

AI-Powered Assistant for Churn Prediction

Classical machine learning, local open-source language understanding, and a marketing-ready conversational interface.

PROJECT	ChurnSignal
DATA	7,043 customers, 21 original variables
FINAL MODEL	30% raw Logistic Regression + 70% feature-engineered CatBoost
ACTION THRESHOLD	0.31, selected from training-only out-of-fold predictions
LOCAL LLM	Ollama with Qwen3 1.7B
DELIVERY	FastAPI, React UI, single and bulk scoring, PDF/Excel outputs
DATE	2026-08-22

Purpose

This report explains how the repository satisfies the assessment: how the data is cleaned, why each evaluation control exists, why the ensemble was selected, how natural language becomes validated model input, and how the result reaches the marketing user.

The report is intentionally concise. The executed notebook remains the full experimental audit trail, while the root README contains operating instructions and the complete figure index.

Technical and business motivation

The design starts from the marketing decision that the model must support, then chooses the simplest controls that make that decision credible.

Business motivation

The client needs more than a notebook probability. Marketing needs to identify customers who deserve attention, understand how the score compares with an intervention threshold, and receive a practical next action. Missing a real churner is assumed to cost more than contacting some customers who would have stayed, so recall is prioritized while F1 is kept close to the strongest candidate.

Business need	Design response	Operational value
Prioritize likely churners	Continuous risk plus Churn/No Churn at 0.31	Supports ranking and campaign-list creation
Reduce missed churn	Recall-oriented hybrid selection	Catches about 76% of holdout churners
Serve non-technical marketers	Conversation, clarification, and 19/19 readiness	Removes manual JSON construction
Make output actionable	Threshold distance, signals, action, and PDF	Connects prediction to retention review
Protect local PoC data	Qwen3 1.7B through local Ollama	No hosted LLM or third-party prompt transfer
Support campaign lists	CSV/XLSX scoring with workbook and PDF	Adds auditable portfolio triage

Technical motivation

Language understanding and prediction are separated. Deterministic rules and a schema-constrained local LLM translate explicit facts; Pydantic validates them; the frozen Logistic/CatBoost ensemble alone computes churn. Stratified CV and out-of-fold probabilities preserve data efficiency and prevent the final test from influencing tuning, blending, or threshold selection.

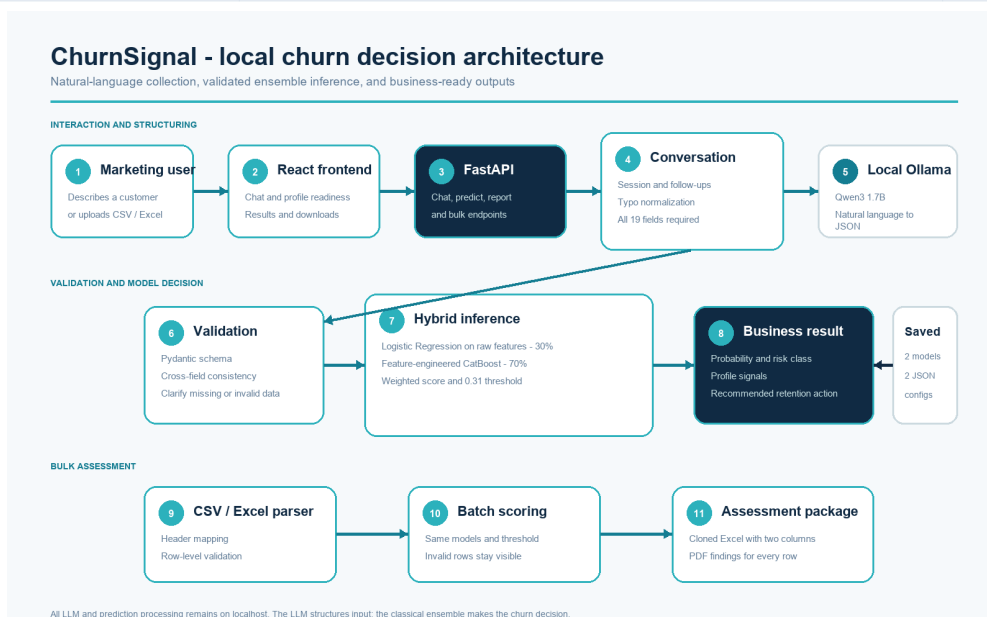
- Logistic Regression supplies interpretable additive structure; CatBoost supplies non-linear categorical interactions.
- A 30/70 probability blend is selected from training-only OOF evidence, not from the holdout.
- The same saved artifacts and feature contract serve JSON, chat, reports, and bulk files.

Hosted LLMs, LLM-based churn prediction, RAG, databases, Kubernetes, feature stores, and automated retraining are excluded because they add privacy, cost, or operational complexity without being necessary to demonstrate this PoC.

How the solution meets every stated requirement

Each item from the client brief is mapped directly to implementation evidence and a delivery status.

Client requirement	Implementation and evidence	Status
Classical churn model	Five model families compared; tuned Logistic/CatBoost hybrid; notebook and models/	Met
Use supplied dataset	7,043-row CSV audited, cleaned, modeled, and held out for final test	Met
Explain model, features, training	Executed notebook, README rationale, comparison tables, and two reports	Met
Conversational input pipeline	Stateful ChatService asks until all 19 validated inputs are present	Met
Open-source, non-third-party LLM	Qwen3 1.7B runs locally through Ollama	Met
Translate language to structure	Deterministic normalization plus Pydantic-constrained JSON patches	Met
Clear understandable output	Risk, probability, threshold, profile signals, action, and PDF	Met
Deploy as API	FastAPI health, predict, chat/reset, report, and bulk endpoints	Met
Architecture diagram	PNG/SVG/Mermaid source in diagram/ and diagram below	Met
Technical/business motivation	Section 01 plus model-selection and business-meaning sections	Met
Requirement alignment	This explicit requirement-to-evidence matrix	Met
Simple suitable tools	Local Ollama + FastAPI + React; no unnecessary cloud/MLOps stack	Met



The LLM structures facts; the classical ensemble makes the churn decision. Optional bulk scoring and professional reports extend marketing usefulness without replacing a mandatory deliverable.

Tools and libraries

The stack remains local and lightweight; the table lists material dependencies rather than every transitive notebook package.

Area	Tool/library	Purpose and selection reason
Analysis	Jupyter, pandas 2.3.3, NumPy 2.0.2	Auditable experiments, cleaning, frames, and vectorized transformations
Statistics/plots	SciPy 1.13.1, Matplotlib 3.9.4, Seaborn 0.13.2	Non-parametric tests, EDA, threshold plots, and confusion matrices
Classical ML	scikit-learn 1.6.1	Pipelines, preprocessing, CV, tuning, baselines, Logistic/tree models, and metrics
Boosting	CatBoost 1.2.10, XGBoost 2.1.4	Boosted-family comparison and selected categorical non-linear model
Persistence	joblib 1.5.3, CatBoost format, JSON	Preserves both fitted models, features, weights, and threshold
API	FastAPI 0.128.8, Uvicorn 0.39.0	Typed lightweight REST service, ASGI runtime, and OpenAPI docs
Validation	Pydantic 2.13.4	Exact enums, ranges, dependencies, and structured LLM output
Local LLM	Ollama + Qwen3 1.7B, HTTPX 0.28.1	Private local extraction with asynchronous HTTP calls and timeout handling
Bulk files	openpyxl 3.1.5, python-multipart 0.0.20	Excel recreation and multipart CSV/XLSX upload
Reports	ReportLab 4.4.3, pypdf 6.0.0	Local deterministic PDF creation and document verification
Backend QA	pytest 8.4.2	API, schema, chatbot, correction, report, and bulk regression tests
Frontend	React 19.2.6, TypeScript 5.9.3, vinext, Vite	Typed stateful chat/dashboard UI and production build
Frontend QA	ESLint 9.39.4, Node test runner	Static checks and rendered-content smoke tests
Architecture	Mermaid with PNG/SVG exports	Editable source plus presentation-ready documentation assets

Why the stack is intentionally small

FastAPI/Uvicorn is sufficient for two compact CPU-friendly models. Ollama satisfies the open-source local-LLM constraint. React supports the stateful UI. A model server, RAG framework, vector store, database, Docker/Kubernetes, MLflow, or feature store would add components without improving the required demonstration. They become relevant only when concurrency, durable history, deployment repeatability, model governance, or monitoring is required.

Exact Python versions are pinned in requirements.txt; the frontend dependency graph is locked in frontend/package-lock.json.

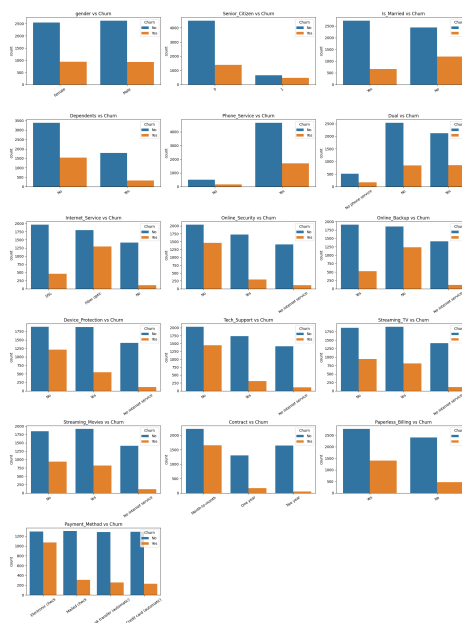
Data audit, EDA, and leakage controls

The notebook validates the table before modeling and keeps the final holdout isolated from every design decision.

Dataset and cleaning

The source contains 7,043 customer rows and 21 original columns. Customer_ID is retained only for traceability, Churn is the target, and the remaining 19 fields form the inference schema. Churn prevalence is 26.54%, so accuracy alone is unsafe.

Step	What is done	Why
Header normalization	Strip whitespace	Prevents hidden schema mismatches
Total_Charges	Parse numeric; fill 11 blank zero-tenure rows with 0	Uses a domain-supported value instead of blind median filling
Integrity	Check missingness, duplicates, IDs, categories	Protects row meaning and prevents identifier leakage
Dependencies	Validate phone and internet service combinations	Rejects impossible customer profiles
EDA	Distributions, target slices, correlations, effect sizes	Reveals imbalance, skew, segment differences, and useful relationships



Why these statistical tests

Chi-square checks categorical dependence while Cramer's V reports effect magnitude. Mann-Whitney U compares skewed numeric groups without a normality assumption, and rank-biserial correlation reports direction and size. Spearman correlation is used for monotonic numeric relationships that need not be perfectly linear.

Leakage control

A stratified 80/20 split creates 5,634 training and 1,409 test rows. Encoding and scaling live inside pipelines, so each cross-validation fold fits its own transformations. The test labels are not used for tuning, feature selection, ensemble weighting, or threshold selection.

Model search and feature engineering

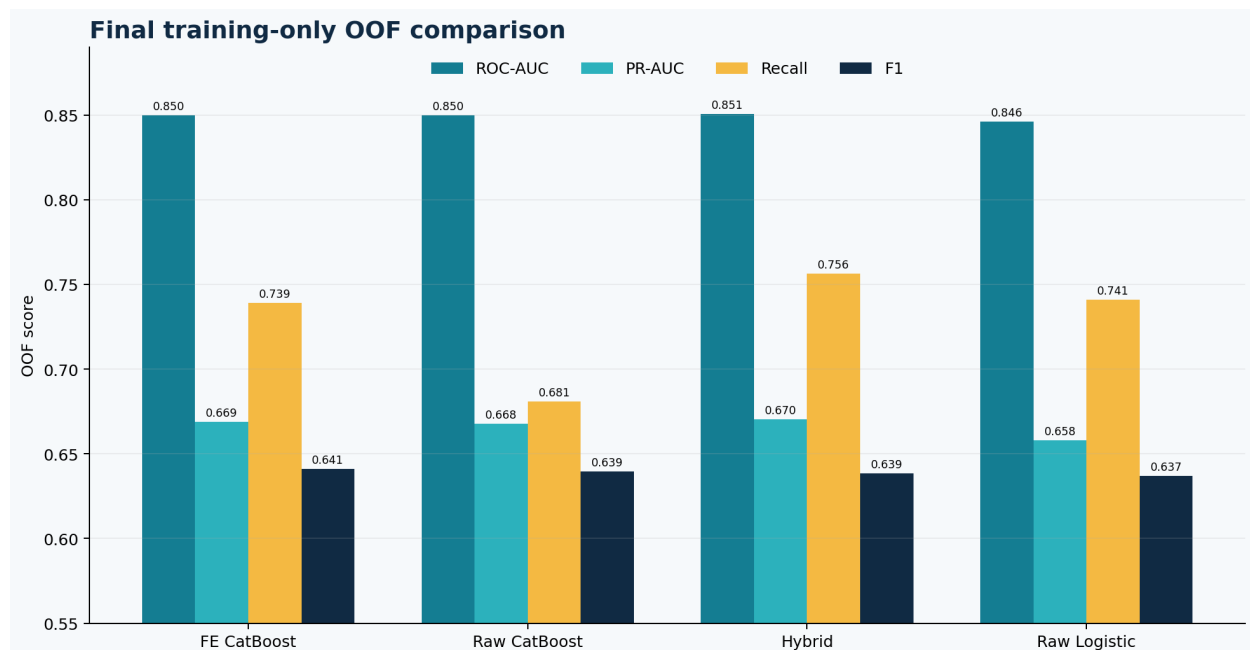
The final model is the result of visible baseline, family-comparison, tuning, and feature-engineering trials.

Experiment logic

- The most-frequent baseline proves that 73.46% accuracy can still mean zero churn recall.
- The unscaled Logistic trial exposes convergence warnings; numeric scaling then improves optimization stability.
- Decision Tree, Random Forest, XGBoost, one-hot CatBoost, and Logistic Regression test different model families before tuning.
- Native CatBoost is revisited because ordered categorical handling is more appropriate than judging it only after one-hot expansion.
- Grid search is used for the small Logistic space; randomized search limits CatBoost compute while exploring more combinations.

Engineered features

Feature	What it summarizes	Why it may help
Num_Services	Six active add-ons	Product depth and switching friction
Num_Support_Services	Four protective services	Support coverage
Num_Streaming_Services	TV and movies	Entertainment engagement
Is_New_Customer	Tenure <= 6 months	Early-life churn exposure
Auto_Payment	Automatic bank/card payment	Lower billing friction
No_Security_And_No_Support	Joint absence of both	Explicit risk interaction



All displayed comparison metrics are produced from training-only out-of-fold probabilities. Every row is scored by a fold model that did not train on it, allowing fair weight and threshold selection without consuming the holdout.

Why the hybrid ensemble is preferred

The decision is driven by business risk, not by claiming one model wins every metric.

Candidate	ROC-AUC	PR-AUC	Threshold	Precision	Recall	F1
FE CatBoost	0.8500	0.6688	0.33	0.5661	0.7391	0.6411
Raw CatBoost	0.8498	0.6677	0.38	0.6027	0.6809	0.6394
Hybrid	0.8505	0.6705	0.31	0.5525	0.7565	0.6386
Raw Logistic	0.8459	0.6581	0.33	0.5582	0.7411	0.6368

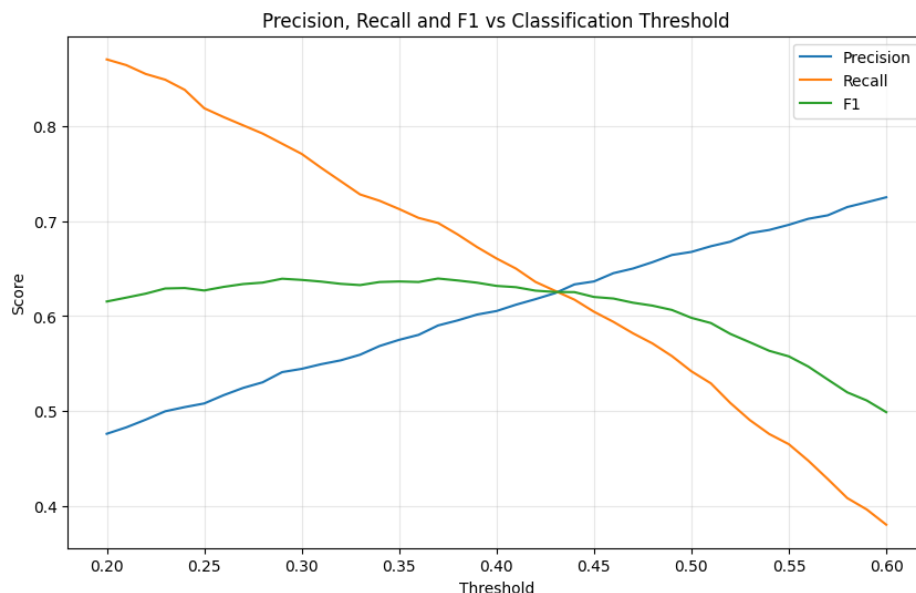
The hybrid model is currently the preferred candidate from a business-risk perspective because it achieves substantially higher recall while maintaining nearly the same F1 score. However, the final production choice should depend on the actual business cost of false negatives versus false positives.

What the comparison means

- Feature-engineered CatBoost has the highest displayed F1 by a small margin.
- Raw CatBoost has the highest precision and would create a smaller, cleaner campaign list.
- The hybrid has the highest recall, ROC-AUC, and PR-AUC, so it catches more potential churners and ranks them slightly better.
- The hybrid therefore fits the working assumption that missing a churner is more expensive than contacting some customers who would stay.

Why 30% Logistic and 70% CatBoost

OOF weight search shows that CatBoost should provide most of the probability because it captures non-linear service and contract interactions. Logistic Regression retains a smaller contribution because it captures smooth, stable global structure. The blend uses complementary error patterns rather than averaging for its own sake.

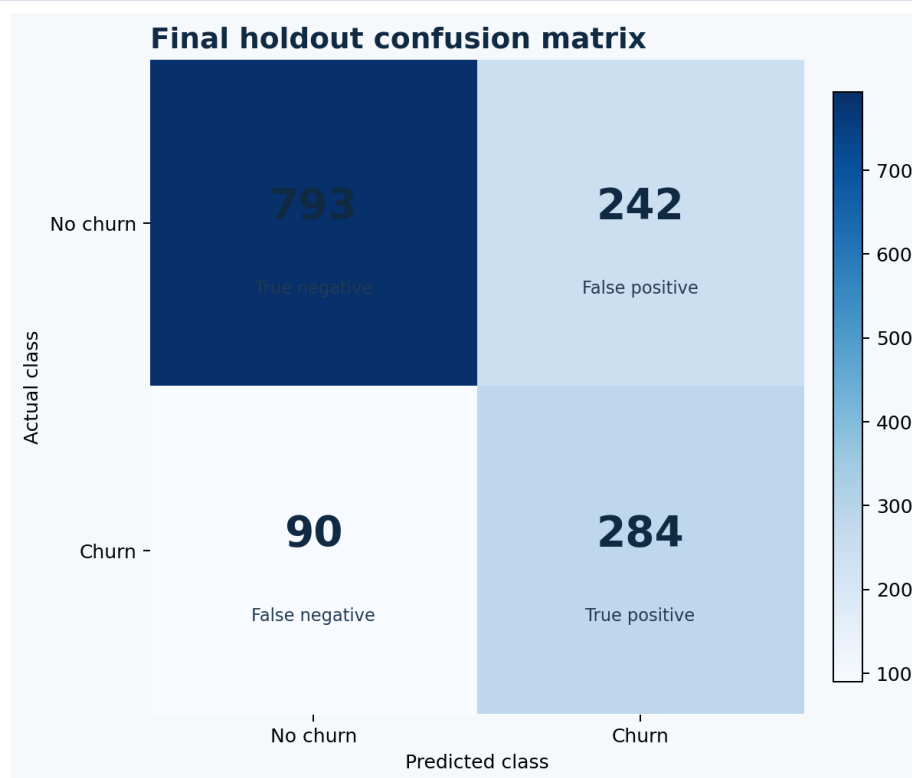


Lower thresholds increase recall and false positives; higher thresholds increase precision and missed churn. The 0.31 operating point is frozen before final test evaluation.

Final performance and business meaning

The frozen ensemble is evaluated once on 1,409 untouched customers.

Metric	Value	Marketing interpretation
Accuracy	76.44%	Overall label correctness
Precision	53.99%	About 54 of every 100 contacted customers actually churn
Recall	75.94%	About 76 of every 100 actual churners are caught
F1	63.11%	Balance of precision and recall at the 0.31 threshold
ROC-AUC	84.65%	Strong ranking across all possible thresholds



Operational reading

There are 374 actual churners. The model catches 284 and misses 90. It flags 526 customers for intervention: 284 true positives and 242 false positives. For a campaign owner, this is clearer than reporting AUC alone.

ROC-AUC 0.8465 is a ranking measure, not proof that every probability is calibrated. Until calibration is measured, the percentage should be described as a model-estimated churn risk.

Chatbot, API, and bulk delivery

The interface collects usable model data while preserving deterministic ML inference.

Conversation lifecycle

Step	Behavior	Why
1. Extract	Rules recover high-confidence shorthand and typos; Ollama supplies schema-constrained fields	Improves speed while retaining natural-language flexibility
2. Merge	Only explicit fields update session state	Prevents the LLM from guessing missing facts
3. Validate	Enums, numeric bounds, and service relationships are checked	Stops malformed profiles before inference
4. Clarify	The assistant requests only missing or conflicting fields	Keeps the conversation short and complete
5. Predict	Both saved models score the validated profile	Reuses the exact notebook inference contract
6. Explain	Probability, threshold position, signals, and next action are returned	Makes the result usable by marketing

API surface

Endpoint	Purpose
GET /health	API and Ollama readiness
POST /predict	Structured single-customer score
POST /chat	Stateful natural-language collection and score
DELETE /chat/{session_id}	Conversation reset
POST /report	One-customer assessment PDF
POST /bulk-predict	CSV/Excel scoring and ZIP delivery

Bulk path

CSV and Excel uploads are limited to 15 MB and 20,000 rows. Known header aliases are mapped deterministically; Ollama is consulted only when headers remain unfamiliar. Each row validates independently, valid rows score in one vectorized call, and invalid rows remain visible as errors. The returned ZIP contains a cloned workbook with Churn and Churn_Percentage plus a row-level findings PDF.

Saved inference contract

The Logistic pipeline, CatBoost model, ensemble configuration, and feature configuration are all required. Reload parity in the [notebook proves that serialization reproduces the pre-save score](#).

The frontend exposes all 19 profile fields, profile readiness, local-model status, chat, reset, result cards, report download, and CSV/Excel upload. No user data needs to be sent to a third-party LLM API.

Verification, limitations, and handoff

The PoC is complete for the assessment while remaining explicit about what production would require.

Verified evidence

- All 159 notebook code cells are executed, with no persisted error outputs.
- Four inference artifacts are present: two models and two JSON configuration files.
- Backend automated tests: 23 passed.
- Frontend production build completes successfully.
- A live local Ollama conversation was exercised through FastAPI and completed a 19-field prediction.
- Single-customer PDF and CSV/Excel bulk-package generation are covered by backend tests.

PoC limitations

Boundary	Current state	Production next step
Validation time	Random stratified split; no timestamp exists	Validate on a later real customer period
Probability quality	Discrimination measured; calibration not measured	Add calibration curve and Brier score
Threshold economics	Recall-oriented 0.31 working policy	Optimize with campaign cost and customer value
Session storage	In-process memory	Persistent store with expiry and privacy controls
Monitoring	Not required for this local PoC	Track schema, drift, score mix, latency, errors, and delayed labels
Fairness	Not evaluated	Compare error rates and calibration across approved segments

Submission assets

Location	Contents
README.md	Full project guide, numbers, comparisons, run steps, and limitations
diagram/	Architecture PNG, SVG, Mermaid, and diagram notes
imgs/	17 extracted notebook figures plus final comparison and holdout matrix
output/pdf/	This concise report plus the complete repository guide

Conclusion

The repository satisfies the assessment's functional PoC requirements: a classical churn model, a conversational local-LLM input pipeline, understandable outputs, and an API. The selected hybrid favors churn recall with almost unchanged F1, which is appropriate for the stated business-risk assumption. Production adoption should confirm that assumption with real intervention costs and capacity.

The senior ML engineering review influenced the report by separating PoC completion from production concerns: the report does not present Docker, model registries, feature stores, drift systems, or automated retraining as assessment requirements.